

Honey-LLM: An Interactive, "Self-Healing" Honeytrap for Agentic AI

Capstone Project Presentation

Student: Devansh Wadhwani

Mentor: [Insert Mentor Name]

CPG No: [Insert CPG Number]

Project Overview and Need Analysis

- **The "Smart Mirror" Trap:** Honey-LLM is a proactive decoy designed to fight AI with AI. It acts as a Dynamic Deception Environment, replacing vulnerable entry points with a sandboxed Large Language Model (LLM).
- **The Shift:** In 2025-2026, AI threats evolved from simple chatbots generating "bad words" to **Agentic AI** capable of executing "bad actions" autonomously at machine speed.
- **The "Shadow Trust" Gap:** Traditional firewalls cannot understand "intent." They allow legitimate-looking API calls to pass through, even if triggered by an attacker hijacking an internal AI agent via Indirect Prompt Injection.

Related Work / Literature Survey

- **The Threat:** The OWASP Top 10 for LLMs identifies Prompt Injection as the #1 critical vulnerability (OWASP, 2025). Attacks on Agentic Assistants require new paradigms (*arXiv:2412.01239*).
- **The Shortcoming:** Traditional defenses fail against semantic chaining. Studies on LLM vulnerability (*PMC11227090*) show proactive defense is mandatory.
- **Generative Deception:** Recent papers validate our approach. The "LLM Agent Honeytrap" (*arXiv:2410.11985*) and "Beekeeper" (*IEEE: 10508681*) showcase using LLMs for interactive trapping, proving that active deception outperforms passive blocking.

Problem Statement and Objectives

Problem: Current security systems fail to distinguish legitimate prompts from malicious Agentic AI jailbreaks in real-time.

Objectives:

1. **Analyze:** Develop a machine learning classifier to detect malicious intent by analyzing the "vibe" of a prompt rather than searching for predefined keywords.
2. **Deceive:** Implement a "Counter-Hallucination Engine" utilizing a local LLM to dynamically generate realistic, fake secrets (e.g., passwords) to maximize attacker dwell time.
3. **Harden:** Create an Autonomous Feedback Loop that translates trapped attacker techniques into automated guardrails (WAF Rules/MCP filters).

Assumptions and Constraints

- **Constraint 1 (Latency):** The intent classification and switch to the decoy sandbox must occur with extremely low latency (< 2 seconds) to remain believable.
- **Constraint 2 (Resource Overhead):** Running conversational SLMs and real-time inference requires strict management of GPU and memory to prevent host system instability.
- **Assumption 1 (Dataset Validity):** The intent classifier relies heavily on the availability of up-to-date adversarial benchmark datasets (e.g., JailbreakBench).
- **Assumption 2 (Zero-Trust):** Attackers will attempt to escape; therefore, the deceptive LLM must be fully isolated within Docker containers to mitigate "break-out" risks.

Project Execution Plan (Broad Methodology)

1. **Establish Sandbox:** Create a Zero-Trust environment using Docker.
2. **Deploy Decoy Persona:** Setup a Small Language Model (e.g., Llama 3) as a vulnerable system (e.g., "Corporate Assistant") via complex system prompts.
3. **Train Intent Classifier:** Train an ML model (DistilBERT) to dynamically analyze prompts and calculate the Kullback-Leibler (KL) Divergence against a baseline "Safety Distribution."
4. **Develop Routing Logic:** If anomalies (malicious intent) are detected, silently redirect the user from the real database to the Honey-LLM sandbox.
5. **Generative Deception & Logging:** The Honey-LLM actively lies to the attacker, logging their methods, which are then autonomously converted into JSON guardrails.

Project Requirements

Tools, Platforms, and Hardware

- **AI/ML Layer:**
 - Generative Engine: Llama-3-8B / Phi-3 (via Ollama/vLLM)
 - Intent Classifier: DistilBERT / DeBERTa-v3
- **Backend & Interface:** FastAPI (router), SQL Database (logging)
- **Infrastructure:** Docker (Containerization and isolation)
- **Datasets:** HuggingFace datasets (HackAPrompt, JailbreakBench)
- **Testing Tools:**
 - Red-Teaming frameworks: Garak, PyRIT
 - Guardrail software: Nvidia NeMo / LLM-Guard
- **Hardware:** Local GPU required for SLM fast inference

Project Outcomes

- A functional, low-latency (<2s) Generative AI honeypot capable of trapping automated hacking bots in a believable simulation.
- A highly accurate Machine Learning classifier that intercepts advanced prompt injection constraints and jailbreak attempts in real-time.
- Tangible demonstration metrics highlighting increased "**Attacker Dwell Time**" alongside high "**Detection Accuracy.**"
- A self-healing pipeline that automatically hardens production applications via deployable WAF rules based on the decoy's threat intelligence.

Work Plan (Months 1-3)

- **Month 1 (Foundation & Architecture):**
 - Docker sandboxing setup
 - Ollama/vLLM installation and SLM latency testing
 - FastAPI backend setup
- **Month 2 (The "Identity" & MVP):**
 - System prompt engineering for the AI persona
 - Creation of static deception files (fake CSVs/JSONs)
 - SQL interaction logging integration
- **Month 3 (The Intelligence Layer):**
 - Gathering adversarial datasets from HuggingFace

Work Plan (Months 4-6)

- **Month 4 (Generative Deception):**
 - Programming the deception switch logic
 - Building the dynamic hallucination engine to generate fake secrets
- **Month 5 (Automated Defense & Testing):**
 - Developing scripts to turn attack logs into real JSON guardrails
 - Conducting stress tests using automated red-teaming (Garak/PyRIT)
- **Month 6 (Documentation & Finalization):**
 - Compiling performance metrics (Dwell Time vs. Accuracy)
 - Final thesis on Adversarial Machine Learning
 - Live dashboard demo preparation

Role / Contributions of Individual Team Members

Devansh Wadhwani

- **Machine Learning Integration:** Lead Developer for the ML Intent Classifier and KL Divergence mathematical implementation.
- **System Architecture:** Architected the Docker Sandbox isolation strategy and FastAPI real-time request routing logic.
- **Deception Logic:** Engineered the SLM System Prompts and interactive Hallucination Deception mechanism.

(Adjust roles appropriately if team members change)

References

- **Generative Deception:** *"LLM Agent Honeypot: Monitoring AI Hacking Agents in the Wild"* (arXiv:2410.11985, 2024) & *"Beekeeper: Accelerating Honeypot Analysis With LLM-Driven Feedback"* (IEEE 10508681, 2024).
- **Threat Landscape:** *"Prompt Injection Attacks on Agentic Coding Assistants"* (arXiv:2412.01239, 2024) and OWASP Top 10 for LLM Applications (2025).
- **Datasets:** "HackAPrompt" & "JailbreakBench" (HuggingFace).
- **Tools:** NVIDIA NeMo Series & Garak Automated Red-Teaming Framework.